

chapter2

제2장 공공 실시간 데이터셋 이해

1장에서 데이터 엔지니어링과 MLOps를 “운영 가능한 데이터 공급망”과 “노후화하는 ML 제품의 운영”으로 정의했다면, 이 장은 그 공급망에 유입되는 첫 재료 — 공공 실시간 데이터 — 의 실제 모습을 직접 검토한다. 민원·재난·교통·환경 데이터는 깨끗하게 정돈된 표가 아니라, 서로 다른 속도로 유입되고 서로 다른 방식으로 실패하는 끊임없이 변화하는 흐름이다. 본 장은 이 데이터를 세 관점으로 분석한다 — 무엇이 들어오는가(유형), 어떻게 들어오는가(유입 패턴), 왜 실패하는가(품질 위험). 특히 실시간 데이터에서 반복적으로 문제가 되는 시간의 두 종류(이벤트 시간과 처리 시간)를 구분하고, 데이터가 시간에 따라 변하는 현상(드리프트)을 공공 맥락에서 식별한다. 실습(★★☆☆☆)에서는 실제 에어코리아 응답을 파싱해 탐색적 분석(EDA)을 수행하고, 두 시각의 분포를 KS 검정으로 비교해 “값은 변해도 분포 변화가 통계적으로 유의하지 않을 수 있다”는 결과를 실데이터로 관찰하고 그 해석 절차를 익힌다. 이 장에서 분류하는 위험들은 추상적 경고가 아니라, 뒤 장의 실습에서 실제로 발생하고 실측으로 기록된 사건이다 — 그 사건들을 미리 제시하며 진행한다.

학습 목표

공공 실시간 데이터를 형식·유입 패턴·품질 위험의 세 축으로 분류하고, 분류가 곧 운영 설계의 분기 조건임을 설명한다.

JSON 이벤트 로그를 파싱해 탐색적 분석을 수행하고, 필드별 결측을 구분해 점검한다.

이벤트 시간과 처리 시간을 구분하고, 혼동이 만드는 집계 왜곡을 설명한다.

데이터 드리프트를 정의하고 공공 데이터의 원인(정책·계절·이벤트·시스템)으로 분류하며, KS 검정 결과를 해석한다.

민간 현장의 실시간 데이터 방법을 공공 맥락에 적용한다.

2.1 공공 실시간 데이터의 유형

학습 목표

공공 실시간 데이터를 민원/재난/안전/교통/환경으로 분류하고, 각 유형의 형식·유입 패턴·품질 리스크를 연결한다.

왜 “형식”만이 아니라 “유입 패턴과 리스크”로 분류해야 하는지 설명한다.

2.1.1 “실시간”은 하나의 속도가 아니다

실시간 데이터를 처음 다루는 사람은 “실시간 = 빠르게 계속 들어오는 것”이라는 단일 이미지를 떠올리기 쉽다. 그러나 운영 설계의 관점에서 “실시간”은 최소 네 가지 서로 다른 유입 패턴을 뭉뚱그린 말이며, 각 패턴은 다른 설계를 요구한다.

지속 유입(steady stream)은 대체로 일정한 속도로 끊임없이 들어오는 형태다 — 예컨대 업무시간 동안 접수되는 민원이 여기에 가깝다. **버스트(burst)**는 평소에는 유입이 적다가 특정 사건에 폭증하는 형태로, 재난 문자 발송 직후 집중되는 신고가 전형이다. **고빈도(high-frequency)**는 초 단위로 대량 유입되는 형태로, 교통 센서 스트림이 여기에 해당한다. **주기 갱신(periodic update)**은 정해진 간격으로 한 묶음씩 갱신되는 형태로, 분·시 단위로 게시되는 환경 측정값이 대표적이다. 이 구분이 왜 중요한가. 버스트를 전제하지 않고 평균 유입량에 맞춰 처리 용량을 산정하면 사건 당일 시스템이 포화하고, 주기 갱신 데이터를 초 단위로 폴링(polling)하면 쿼터만 소진하며 새 데이터는 없다. 즉 유입 패턴은 처리량 산정·폴링 주기·버퍼 크기라는 구체적 설계 결정으로 직결된다.

이 관점은 스트리밍 시스템 이론과도 일치한다. Akidau 외(2015)의 Dataflow 모델은 데이터를 “끝이 있는(bounded)” 배치와 “끝이 없는(unbounded)” 스트림으로 나누고, 스트림은 본질적으로 순서가 어긋난 채(out-of-order) 도착한다고 전제한다. 공공 실시간 데이터는 대부분 unbounded이고 out-of-order다 — 이 전제를 받아들이는 것이 2.3(이벤트 시간)과 2.4(드리프트)의 출발점이다. (적용 자격: 이 이론은 스트리밍 일반을 대상으로 하며 공공 부문의 직접 선례는 아니다. 개념·방법을 차용한다.)

2.1.2 네 가지 유형의 해부

유형을 형식만으로 나누면 “JSON이나 센서 스트림이냐”에서 멈춘다. 운영에서 사용하는 분류는 형식·유입 패턴·대표 품질 리스크·운영 포인트를 한 줄로 묶는다.

표 2.1 공공 실시간 데이터 유형 비교

유형	형식	유입 패턴	대표 품질 이슈	운영 포인트
민원	JSON 이벤트 로그	업무시간 지속 유입	개인정보, 코드 불일치	마스킹·접근 통제, 코드 매핑
재난·안전	JSON/XML 피드	이벤트 버스트	중복 경보, 위치 오류	피크 처리량, 지연 이벤트 처리
교통	센서 스트림	초당 고빈도	센서 오작동, 통신 지연	실시간 집계, 결측 구간 처리
환경	시계열 측정값	분 단위 주기 갱신	결측, 단위 불일치	품질 모니터링, 표준화

이 표를 관통하는 한 축이 “사람이 입력하는가, 기계가 입력하는가”다. 민원 데이터는 사람이 자유 텍스트로 입력하므로 필드 누락과 표현의 다양성이 자연스럽게 발생하고, 본문에 개인정보가 섞여 들어온다 — 그래서 마스킹과 접근 통제가 수집 단계의 설계 입력이 된다. 반면 교통·환경 센서는 기계가 신호를 보내므로 개인정보 위험은 낮지만, 장비 오작동과 통신 장애가 결측·이상치의 주된

원인이 된다. 재난·안전 데이터에는 그 중간에서 버스트라는 고유의 문제가 있다 — 하나의 사건에 대해 여러 경로로 중복 경보가 들어오고, 위치 정보가 어긋나기 쉽다. 요컨대 같은 “결측”이라도 민원(사람의 누락)과 환경(장비의 장애)은 원인이 다르고, 따라서 대응도 다르다.

2.1.3 분류는 운영 대응의 분기 조건이다

유형 분류의 실무적 의미는 라벨을 붙이는 데 있지 않고, **대응 전략을 미리 구분하는 데 있다**. 버스트형(재난)에는 피크를 흡수할 큐와 지연 이벤트 처리 규칙(6장의 watermark)이 필요하고, 고빈도형(교통)에는 실시간 집계와 결측 구간 보간이 필요하며, 개인정보 결합형(민원)에는 마스킹·접근 로그(13·15장)가 필요하다. 유형을 먼저 판정하지 않으면 대응책을 사후에 개별적으로 추가하게 되고, 파이프라인의 복잡도가 증가한다.

민간 비즈니스 로직의 공공 적용

이 유입 패턴들은 민간 실시간 현장에서 이미 정교하게 다뤄져 왔고, 공공에 적용할 수 있다. 광고·커머스에서 노출·클릭 이벤트를 실시간 집계하는 시스템은 캠페인 시작·세일 시각에 발생하는 트래픽 버스트를 흡수하도록 설계된다 — 이 버스트 흡수 설계가 그대로 재난 신고 폭증 대응으로 옮겨진다. 추천 시스템이 사용자 행동을 초 단위로 받아 피처를 갱신하는 고빈도 파이프라인은 교통 센서 스트림의 실시간 집계와 구조가 같다. 다만 공공에서는 변형이 추가된다: 민간의 실시간 대시보드가 매출·트래픽을 제시한다면, 공공의 현황판은 같은 실시간성 위에 형평 축(지역·집단별 분해)과 감사 가능성을 추가해야 한다. 방법은 같고, 제약이 다르다.

분류 기준에도 차이가 있다. 민간에서는 고객 행동·거래·센서·마케팅으로 데이터를 나누면서 유형별 비용과 수익 기여를 기준에 함께 넣는다. 공공에서는 유형별 감사 의무와 개인정보 통제가 법으로 정해져 있어, 그 통제 수준이 분류의 기준으로 먼저 들어온다. 그래서 같은 데이터라도 민간에서는 “얼마를 벌여 주는 데이터인가”가, 공공에서는 “어느 수준의 통제를 받아야 하는 데이터인가”가 유형을 결정한다.

공공 사례: 같은 “폭증”이 유형에 따라 다른 설계를 부른다

서울시가 초미세먼지 비상저감조치를 발령한 날을 가정한다. 이 하루에 세 유형의 데이터가 동시에 급증한다. 환경 데이터(주기 갱신)는 값의 수준이 올라가지만 유입 구조는 그대로다 — 측정소 40개가 여전히 정시마다 한 묶음씩 전송한다. 반면 관련 민원(지속 유입)은 “매연 차량 신고”·“공사장 비산먼지” 같은 유형의 건수가 늘며 유형 분포가 바뀐다. 그리고 저감조치 안내 문자가 발송되면 그 직후 문의가 재난·안전 채널로 집중된다. 세 유형은 같은 “폭증”을 겪지만 대응이 다르다 — 환경은 값의 드리프트 감시(2.4), 민원은 유형 분포 변화 추적, 재난·안전 채널은 피크 처리량 확보다. 유형을 구분하지 않고 “오늘 데이터가 많다”로 처리하면 세 대응 중 어느 것도 정상적으로 수행되지 않는다. ### 핵심 정리 - “실시간”은 하나의 속도가 아니라 지속·버스트·고빈도·주기 갱신의 서로 다른 유입 패턴을 포함한다. - 유형 분류는 “형식”만이 아니라 “유입 패턴과 품질 리스크”를 포함해야 하며, 그 자체가 운영 대응의 분기 조건이다. - 같은 문제(결측/이상치)라도 데이터 유형에 따라 원인과 대응이 달라진다.

2.2 데이터 품질 위험: 중복, 지연, 결측, 스키마 변경, 개인정보

학습 목표

실시간 데이터에서 빈번한 위험(중복/지연/결측/스키마 변경/개인정보)을 각각 정의→왜→시나리오→현장 방법으로 이해한다.

각 위험을 운영 신호와 최소 대응, 그리고 이 책 뒤 장의 실측 사건으로 연결한다.

실시간 데이터에서 품질 문제는 예외가 아니라 정상 상태에 가깝다. 이 절은 다섯 위험을 하나씩, “무엇인가 → 없으면 무엇이 실패하는가 → 어떤 상황에서 발생하는가 → 현장은 어떻게 방어하는가”의 순서로 서술한다. 각 위험이 이 책 어느 장에서 실제로 발생했는지도 함께 제시한다.

2.2.1 중복: 재전송으로 발생하는 추가 이벤트

중복은 같은 이벤트가 두 번 이상 수신되는 현상이다. 대부분 버그가 아니라, 유실을 막으려는 안전장치(재시도·재처리)의 부작용으로 발생한다. 중복 제거 절차가 없으면 집계값이 실제보다 커진다. 민원 1건이 두 번 세어지면 “이 지역 민원이 늘었다”는 잘못된 신호가 대시보드와 모델 입력에 동시에 들어간다.

구체 시나리오는 다음과 같다. 네트워크 순간 오류로 응답을 받지 못한 수집기가 재시도를 하는데, 사실 서버는 첫 요청을 이미 처리한 상태였다면 같은 데이터가 두 번 저장된다 — “응답을 못 받았다”와 “서버가 처리하지 않았다”는 다르기 때문이다. 이것이 추상론이 아니라는 사실은 4장의 실측에서 확인할 수 있다: 같은 이벤트 스트림을 Kafka로 수집하면서 커밋 시점 한 줄만 바꾸면, 처리를 커밋보다 앞세운 설계(at-least-once)는 크래시(crash) 후 재시작 시 이미 처리한 10건을 다시 읽어 **중복 10건**을 생성했다. 이는 버그가 아니라 전달 보증의 정의대로의 동작이다.

현장의 방법은 **멱등키(idempotency key)**다 — 이벤트마다 고유 식별자를 부여하고, 저장소의 유니크 제약과 Upsert(update+insert)로 “최소 한 번 전달하되 정확히 한 번만 반영”되게 만든다. 5장이 4장의 실측 중복 스트림을 받아 UPSERT로 흡수하는 것을 실습으로 구현한다 — 재전송이 와도 업무 상태가 바뀌지 않는다.

이 위험을 운영에서 감시하려면 중복을 수치로 계수해야 한다. 민간에서는 수집 전체 건수 중 중복이 차지하는 비율을 지표로 두고, 건수가 곧 금액인 과금 시스템에서는 0.01% 미만을 목표로 설정하는 경우가 많다. 공공에서도 같은 지표를 사용하되 목표를 정하는 근거가 다르다 — 민간은 잘못 청구한 금액이고, 공공은 부풀려진 건수로 배분한 인력과 예산이다.

2.2.2 지연: 늦게 도착하는 과거

지연은 이벤트가 발생 시각보다 늦게 도착하는 현상이다. 없으면 무엇이 실패하는가 — 지연을 고려하지 않으면 이미 마감해 보고한 집계값이 나중에 틀린 것이 된다. 22시 민원 건수를 22:20에 확정했는데 22:25에 지연 민원이 도착하면, 확정 수치가 조용히 어긋나거나 소급 수정으로 보고 신뢰가 저하된다.

지연의 핵심은 시간이 두 종류라는 데 있다 — 사건이 발생한 이벤트 시간과 수집 측이 기록한 처리 시간이 벌어진다(2.3에서 상술). 6장이 이를 실측으로 다룬다: Spark Structured Streaming에서 watermark(지연 허용 규칙)를 10분으로 선언하고 지연 이벤트 두 건을 투입하면, 하나(발생 22:04)는 **수용되어 해당 창**의 집계가 3에서 4로 갱신되고, 다른 하나(발생 21:49, 더 늦게 도착)는 **폐기되어 폐기 카운터(numRowsDroppedByWatermark)가 0에서 1로 증가**한다. 두 판정의 증거가 서로 다른 곳에 남는 것이 실무의 핵심이다 — 수용은 결과에 나타나지만 폐기는 결과 어디에도 나타나지 않으므로, 반드시 엔진 메트릭을 지표로 감시해야 한다.

레이블 도착이 늦으면 모델 성능 계산도 늦어진다. 14장의 통합 시스템에서는 실제 건수(레이블)가 하루 늦게 도착하므로 예측 성능을 다음 날 계산했다. 지연 레이블로 계산한 6건의 **평균절대오차(MAE)는 1.0**이었다. 운영에서는 이벤트 시간과 도착 시간의 차이를 측정하고, 관측된 지연 구간을 기준으로 watermark를 정한다. 폐기 건수도 함께 기록해 지연 분포가 바뀌면 watermark를 재검토한다. 워터마크(Watermark)는 실시간 스트리밍 데이터 처리에서 “지연된 데이터(Late Data)를 어디까지 허용하고 기다려줄 것인가”를 정의하는 기준선(시간 임계값)이다.

2.2.3 결측: 값 결측과 파생 결측은 별개다

결측은 있어야 할 값이 비는 현상이다. 장비 오작동·API 오류·파싱 실패로 발생한다. 없으면 무엇이 실패하는가 — 결측을 0으로 대체하면 평균이 왜곡되고(1장에서 본 “없는 값을 생성하는 것”), 결측률 자체를 지표로 두지 않으면 소스가 조용히 고장난 상태를 놓친다.

구체 사례는 실습 2.1에 있다. 서울 40개 측정소의 한 스냅샷에서 PM10 측정값(pm10Value)은 40개 모두 채워져 있었는데, 파생 등급(pm10Grade)은 **한 측정소에서 비어 있었다**. 값은 있으나 그 값에서 계산되어야 할 등급이 누락된 것 — “값 결측”과 “파생 필드 결측”이 별개라는 사실이 실행데이터에서 확인된다. 그래서 결측 점검은 데이터 전체가 아니라 **필드별로 따로** 해야 한다.

현장의 방법 두 가지를 이 책이 실측으로 다룬다. 하나는 **결측 사유의 분리 집계**다 — 에어코리아 응답의 pm10Flag 같은 플래그는 결측의 사유(점검/통신장애)를 표시하므로, “결측 3건” 대신 “점검 2건, 통신장애 1건”으로 분리해 계수해야 조치가 가능해진다(1장 실습 1.1의 플래그 활용). 다른 하나는 **제외한 레코드를 계수하는 것**이다 — 7장의 배치 정제는 원본 60건을 정제하면서 매핑 54 + 미기재 2 + 미매핑 3 + 중복 제거 1로 총계 보존식을 검증한다. 제외한 레코드를 계수하지 않으면 유실과 정제를 구분할 수 없다.

민간에서는 이 점검을 임계값으로 고정해 둔다. 피처마다 결측 허용치를 정해 놓고, 예컨대 추천 모델 입력 피처의 결측률이 5%를 넘으면 경보를 발생시키고 10%를 넘으면 서빙을 중단한다. 공공에서도 같은 두 단계 임계를 사용할 수 있지만, 멈추는 근거는 매출 손실이 아니라 “틀린 수치로 행정 결정을 내리지 않는다”는 데 있다.

2.2.4 스키마 변경: 중단 없이 조용히 틀린다

스키마 변경은 필드가 추가·삭제되거나 타입의 의미가 바뀌는 현상이다. 없으면 무엇이 실패하는가 — 스키마가 바뀌어도 파싱은 즉시 중단되지 않는다. 없는 필드를 조회하면 파서는 오류 대신 None을 반환할 뿐이고, 집계는 계속 수행되면서 결측률만 조용히 100%가 된다. 1장에서 이름 붙인 **침묵 실패(silent failure)**의 전형이다.

구체 시나리오: 생산 시스템이 필드명을 `pm10Value` 에서 `pm10` 으로 바꾸거나, 민원 유형 코드 체계가 개편되어 새 코드가 유입되기 시작한다. 소비자는 아무 신호도 받지 못한 채 새 코드를 전부 “기타”로 분류하고, 며칠 뒤 모델 입력의 범주 분포가 무너져서야 문제가 드러난다(1장 1.3.2의 실패 연쇄 타임라인). 7장은 이 위험의 방어를 실측으로 구현한다 — 표준 코드 사전에 없는 표기(판교·성남시·김포)를 자동으로 “가장 유사한 자치구”에 강제 매핑하지 않고 **미매핑으로 분리 보고**하며, 총계 보존식이 성립하지 않으면 배치 자체를 실패시킨다.

최신 이론과 현장 경험에서 도출한 해결 방법은 세 가지다. 첫째, **스키마 진화 호환성**이다 — Confluent의 스키마 레지스트리 문서는 스키마 변경을 후방 호환(backward: 새 스키마로 옛 데이터를 읽을 수 있음)·전방 호환(forward: 옛 스키마로 새 데이터를 읽을 수 있음) 등으로 분류하고, 어떤 변경이 소비자 시스템을 중단시키는지 규칙으로 정한다. 둘째, **데이터 계약(data contract)**이다 — 생산자와 소비자가 스키마·의미·품질·변경 절차를 명시적으로 합의해 문서와 코드로 관리하는 관행이다. datacontract.com은 이 관행을 오픈 표준(명세)으로 공개했고, ThoughtWorks 등은 현장 도입 사례를 문서화하고 있다. 셋째, **머신러닝을 위한 데이터 검증**이다 — Breck 외(2019)는 Google 프로덕션(production)의 데이터 검증 시스템(TFDV)에서 스키마를 명시적 산출물로 두고, 유입 데이터가 스키마를 벗어나면(schema skew) 경보하는 구조를 제시한다. 세 방법의 공통은 하나다: **스키마를 암묵적 전제가 아니라 검증 가능한 계약으로 승격시켜, 변경이 침묵하지 않게 만든다.** (적용 자격: Confluent-datacontract는 도구·표준 문서, Breck은 Google 프로덕션 대상 방법론 차용 — 공공 직접 선례는 아니나 방법이 그대로 적용된다.)

계약이 문서에만 머무는지 실제로 작동하는지는 민간에서 **스키마 변경 사전 공지율**로 측정한다 — 생산 팀이 스키마를 바꾸기 전에 소비 팀에 알린 비율이다. 이 비율이 낮으면 계약서가 있어도 소비자는 여전히 사고 뒤에 변경을 알게 된다. 공공 발주에서도 같은 항목을 과업지시서의 변경 통보 조항으로 요구할 수 있다.

2.2.5 개인정보: 수집 단계부터 설계 입력

개인정보 위험은 식별 가능한 값이 데이터에 유입되는 현상이다. 민원 텍스트·신고자 정보처럼 사람이 입력한 데이터에 흔하다. 없으면 무엇이 실패하는가 — 개인정보를 수집 후에 처리하려 하면 이미 로그·백업·하류 시스템으로 확산된 뒤이므로 되돌릴 수 없다. 그래서 마스킹·접근 통제는 “나중에 붙이는 보안 기능”이 아니라 수집 단계의 설계 입력이어야 한다.

13장은 역할·목적 기반 접근 정책에 결정적 순서의 요청 12건을 통과시켜 **허용 8건·거부 4건**을 산출하고, 거부 4건(미등록 역할, 목적 외 값 조회 등)을 모두 접근 로그에 남긴다 — 거부의 기록 자체가 감사 증거다. 15장은 공공 LLM 서비스에서 질의가 생성에 이르기 전 PII(개인식별정보, Personally Identifiable Information)를 마스킹하며, 개인정보가 포함된 질의 2건에서 **마스킹 토큰 3건**을 치환한다(전화번호 정규식이 한글이 붙은 번호를 놓치는 사례까지 실측으로 확인한다).

규제는 이 설계의 근거를 조문 단위로 제공한다. 개인정보 보호법 제18조(목적 외 이용·제공 제한)는 수집 목적 밖의 이용을 제한하므로, 접근 정책에 “누가”만이 아니라 “무슨 목적으로”가 들어가야 하고 목적 외 조회는 거부·기록되어야 한다(13장 구조가 정확히 이것이다). 제23조(민감정보 처리 제한)는 건강 등 민감정보에 한정된 더 강한 통제이므로, 일반 개인정보의 목적 제한 근거로 확장 인용하면 검수에서 지적된다 — 조문의 적용 범위를 정확히 지키는 것이 규제 전환의 기본이다. 상세와 실측 적용은 13·15·16장에서 다룬다.

민간도 같은 지점에 같은 설계를 배치한다. GDPR(EU 일반 데이터 보호 규정)과 개인정보 보호법을 지키려고 PII 마스킹을 수집 단계에 넣는 것이 표준이다. 규제의 이름과 과징금의 크기가 다를 뿐, “퍼진 뒤에는 되돌릴 수 없다”는 전제가 같아서 대응 위치도 같아진다.

표 2.2 품질 위험과 운영 대응, 그리고 이 책의 실측

위험	운영에서의 신호	최소 대응	이 책에서 실제로 발생한 장
중복	동일 이벤트가 반복 수신	멱등키/중복 제거 규칙	4장(중복 10건), 5장(UPSERT 흡수)
지연	이벤트 시간과 수신 시간이 벌어짐	이벤트 시간 기준 집계/watermark	6장(수용 갱신·폐기 카운터), 14장(지연 채점 MAE 1.0)
결측	필드별 결측률 증가	소스 점검 + 필드별·사유별 집계	실습 2.1(등급 1건 결측), 7장(총계 보존)
스키마 변경	코드/단위/필드가 조용히 바뀜	데이터 계약, 스키마 검증, 버전	7장(미매핑 분리 보고)
개인정보	식별 가능 필드 유입	마스킹·목적 기반 접근·거부 로그	13장(거부 4건), 15장(PII 토큰 3건)

현장에서 이 다섯 위험을 하나의 감시 체계로 묶는 최신 관점이 **데이터 관측성(data observability)**이다. Monte Carlo 등이 정리한 다섯 기둥 — 최신성(freshness: 데이터가 제때 갱신되는가)·양(volume: 건수가 정상 범위인가)·스키마(schema: 구조가 바뀌었는가)·분포(distribution: 값이 정상 범위인가)·계보(lineage: 어디서 와서 어디로 가는가) — 은 위 표의 위험들을 “한 번 정제하고 끝”이 아니라 지속 감시하는 지표로 전환한 것이다. (적용 자격: 민간 기술 블로그의 현장 방법 — 공공에서도 지표 정의는 그대로 유효하다.)

민간에서는 이 다섯 지표에 SLA(서비스 수준 합의)를 붙인다. 위반이 곧 매출 손실이므로 임계에 금액을 환산해 부여하고, 초과하면 계약상 책임이 발생한다. 공공에서는 금액 대신 보고 기한과 감사 대응이 임계의 근거가 된다 — 최신성 지표가 기준에 미달하면 정해진 시각에 보고할 수치가 없고, 계보 지표가 없으면 “이 수치가 어디서 왔는가”라는 감사 질문에 답하지 못한다. 지표의 정의는 같고, 임계를 정당화하는 단위가 금액에서 기한·책임으로 바뀐다.

공공 사례

실습 2.1에서 호출한 대기질 데이터처럼 “기계가 보내는” 환경 센서는 개인정보 위험이 낮지만 결측·스키마 위험은 그대로 있다. 실제로 이 한 번의 응답에서도 `pm10Value` 는 모두 채워져 있는데 한 측정소의 등급(`pm10Grade`)만 비어 있었다 — 값과 파생 등급의 일관성이 유지되지 않은 경우다. 반면 민원 데이터는 “사람이 입력하는 텍스트”라 개인정보가 본문에 섞여 들어오므로, 같은 ‘결측’이라도 환경 데이터와 대응이 완전히 다르다. 따라서 위험은 “한 번 정제하고 끝”이 아니라, 변화 신호를 감지하고 원인을 추적할 수 있게 운영화해야 한다.

핵심 정리

품질 문제는 예외가 아니라 실시간 데이터의 정상 상태에 가깝다.

각 위험은 운영 신호(지표)와 최소 대응(규칙/계약/검증)을 함께 설계해야 하며, 뒤 장에서 실측 사건으로 다시 다룬다.

스키마는 암묵적 전제가 아니라 검증 가능한 계약으로 승격시켜야 침묵 실패를 막는다.

2.3 이벤트 시간과 처리 시간의 차이

학습 목표

이벤트 시간(event time)과 처리 시간(processing time)을 구분한다.

둘을 혼동하면 생기는 집계 왜곡을 설명하고, 재현성을 위해 어떤 시간을 기록해야 하는지 안다.

2.3.1 정의와 왜 벌어지는가

이벤트 시간은 “사건이 발생한/측정된 시각”이고, 처리 시간은 “수집 시스템이 그 데이터를 받은/처리한 시각”이다. 이 둘은 이론적으로도 실무적으로도 거의 항상 벌어진다 — 네트워크 지연, 배치 갱신, 재전송, 그리고 원천 자체의 게시 지연 때문이다. 특히 공공 환경 데이터에는 구조적 게시 지연이 있다: 정시(예: 22:00)에 측정된 값이 정시보다 늦게 게시되므로, 로컬 시계가 아니라 응답 내용(`dateTime`)으로 “이것이 몇 시 데이터인가”를 확인해야 한다(11장 수집기가 실제로 이 지연 때문에 폴링을 여러 번 반복한다).

2.3.2 혼동의 대가

없으면 무엇이 실패하는가 — 처리 시간으로 집계하면 “언제 일어난 일인가”가 흐려진다. 실습 2.1의 대기질 응답에서 각 레코드의 `dateTime` 은 측정 시각(예: 2026-06-28 22:00)이다. API를 호출해 수신한 시각은 그보다 늦으며, 이것이 처리 시각이다. 만약 22:30에 받은 데이터를 “22:30의 대기질”로 기록하면 30분의 오차가 그대로 지표에 들어간다. 환경 데이터는 정시 갱신이라 차이가 작아 보이지만, 재난·교통처럼 버스트가 큰 데이터에서는 이벤트 시간과 처리 시간을 구분하지 않으면 피크 시각 자체가 어긋난다 — “몇 시에 신고가 몰렸는가”라는 가장 중요한 질문의 답이 어긋난다.

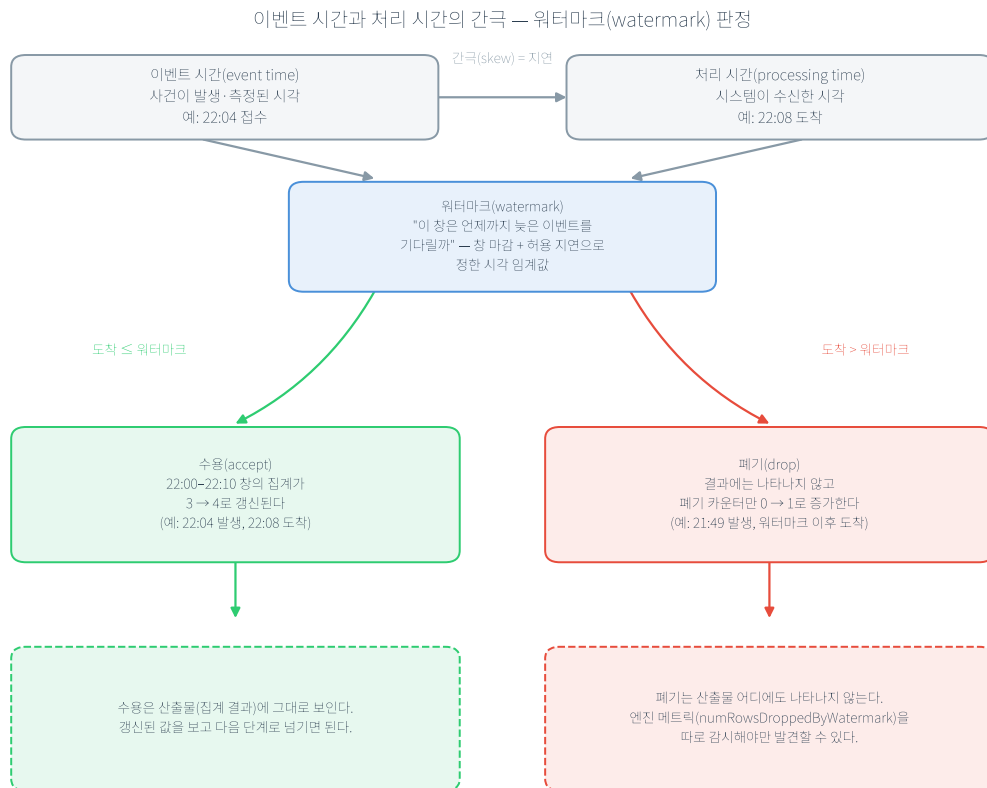
표 2.3 이벤트 시간과 처리 시간의 대비

구분	이벤트 시간(event time)	처리 시간(processing time)
뜻	사건이 발생·측정된 시각	시스템이 수신·처리한 시각
공공 예	민원 접수 시각, 대기질 dateTime	API 응답 수신 시각, 배치 실행 시각
벌어지는 원인	—	네트워크 지연, 배치 갱신, 재전송, 게시 지연
집계 기준으로 사용하면	“언제 발생했는가”가 정확	“언제 발생했는가”가 흐려짐
지연 이벤트 처리	watermark로 수용/폐기 규칙 필요(6장)	규칙 없이 마감 → 소급 수정 위험

표의 마지막 두 행은 설계할 때 선택해야 할 항목이다. 운영 지표를 이벤트 시간 기준으로 설정하면 지연 이벤트를 어떻게 다룰지(수용할지 폐기할지)를 명시적으로 정해야 하고, 처리 시간 기준으로 설정하면 그 결정을 회피할 수 있지만 대신 “몇 시의 통계인가”가 부정확해진다. 공공 보고의 신뢰성을 고려하면 전자가 원칙이며, 그 대가로 6장의 watermark 설계가 따라온다.

2.3.3 이론과 실측

이 구분은 스트리밍 시스템 이론의 핵심 주제다. Akidau 외(2015)의 Dataflow 모델은 이벤트 시간과 처리 시간의 간극(skew)을 스트림 처리의 근본 문제로 놓고, “언제까지 늦은 데이터를 기다릴 것인가”를 **watermark**라는 개념으로 형식화한다. Kleppmann의 *Designing Data-Intensive Applications*도 이벤트 시간 기준 집계와 지연 이벤트 처리를 데이터 시스템 설계의 표준 주제로 다룬다. 이 책은 그 이론을 6장에서 실측으로 확인한다 — watermark를 10분으로 선언했을 때 지연 이벤트가 수용되면 집계가 갱신되고(창 22:00–22:10 강남구 3→4), 폐기되면 결과에는 보이지 않고 오직 메트릭에만 남는다. 이론이 제시한 “간극을 어떻게 타협하는가”의 구체적 사례다. (적용 자격: Dataflow-DDIA는 스트리밍 일반 대상의 방법론·배경 원칙 차용 — 공공 직접 선례는 아니다.)



이벤트 시간과 처리 시간의 간극, 그리고 워터마크 판정

그림 2.1 이벤트 시간과 처리 시간의 간극 — 워터마크(watermark) 판정

그림에는 6장 실측이 두 단계로 나뉘어 제시된다. 위쪽은 이벤트 시간(사건이 발생한 시각)과 처리 시간(시스템이 수신한 시각) 사이의 간극이다. 이 간극을 얼마나 허용할지 정한 값이 워터마크다. 도착 시각이 워터마크보다 이르면 수용하고, 늦으면 폐기한다. 수용된 이벤트(예: 22:04 발생, 22:08 도착)는 해당 창의 집계계를 3에서 4로 갱신하지만, 폐기된 이벤트(예: 21:49 발생, 워터마크 이후 도착)는 산출물에 흔적을 남기지 않고 폐기 카운터만 증가시킨다. 두 처리 결과의 비대칭이 이 판정의 핵심이다 — 수용은 결과를 보면 확인되지만, 폐기는 엔진 메트릭을 따로 감시하지 않으면 발견되지 않는다.

2.3.4 현장의 방법과 공공 적용

원칙은 명확하다: 집계·드리프트 판단은 이벤트 시간 기준으로 설계하고, 산출물에는 어떤 시간을 사용했는지(측정 시각/처리 시각)를 명시해 재현성을 확보한다. 지연 이벤트는 이미 마감한 시간 창의 집계계를 바꿀 수 있으므로 보정 규칙(6장 watermark)이 필요하다.

민간 실시간 현장은 이 문제를 오래 다뤄 왔다. 광고 클릭 로그를 집계할 때, 클릭이 발생한 시각과 로그가 수집 서버에 도달한 시각은 모바일 네트워크 사정으로 크게 벌어질 수 있고, 정확한 시간대별 성과 집계는 반드시 이벤트 시간(클릭 발생 시각) 기준이어야 한다. 이 방법을 공공에

적용하면 민원 접수 시각 기준 집계가 된다 — “22시 대 민원 건수”는 22시에 접수된(이벤트 시간) 건수여야지, 22시에 시스템이 처리한(처리 시간) 건수가 아니다. 방법은 같고, 대상이 매출에서 행정 통계로 바뀔 뿐이다.

민간 현장에서는 이 간극을 두 지표로 고정해 감시한다. 하나는 **이벤트-처리 시간 차이(skew)의 분포**로, p50-p95-p99를 따로 추적한다 — 광고 시스템에서 p99가 30분을 넘으면 일별 정산의 정확도가 떨어진다. 다른 하나는 **지연 이벤트 비율**, 곧 watermark를 넘겨 폐기된 이벤트의 비율이다. 이 비율이 올라가면 watermark 값을 늘려야 한다는 신호로 해석한다. 공공 민원 집계에도 같은 두 지표가 필요하다. 보고 마감 시각을 몇 시로 설정할지 정하려면 “해당 데이터의 지연이 어디까지 벌어지는가”를 수치로 파악해야 하고, 그 수치 없이 정한 마감은 소급 수정으로 이어진다.

공공 사례: 시간 기준이 인력 배치를 바꾼다

구청이 시간대별 민원 인력을 배치한다고 가정한다. 22:00~23:00에 접수된 민원이 실제로는 40건인데, 시스템 부하로 처리(적재)가 지연되어 그중 15건이 23시 이후에 적재됐다면 — 처리 시간 기준 집계는 “22시대 25건”으로 보고하고, 다음 배치가 그 수치로 인력을 정한다. 결과적으로 22시대는 실제보다 적게, 23시대는 많이 배치되어 두 시간대 모두 어긋난다. 이벤트 시간(접수 시각) 기준으로 집계했다면 40건이 온전히 22시대로 귀속되어 배치가 정확했을 것이다. 이처럼 “어느 시간을 기준으로 세는가”는 통계 정확성의 문제를 넘어, 그 통계를 입력으로 삼는 하류 의사결정(인력·예산 배분)까지 바꾼다 — 지연이 큰 데이터일수록 이 차이가 커진다.

핵심 정리

시간에는 두 종류가 있고, 운영 지표는 이벤트 시간 기준으로 설계해야 한다.

이벤트 시간과 처리 시간의 간극은 스트리밍 시스템의 근본 문제이며, watermark로 명시적 규칙이 된다(6장 실측).

산출물에는 어떤 시간을 사용했는지(측정 시각/처리 시각)를 명시해 재현성을 확보한다.

2.4 데이터 드리프트의 직관적 이해

학습 목표

드리프트를 “시간에 따른 데이터 분포 변화”로 정의하고, 왜 위험 신호이자 경보 피로의 원천인지 설명한다.

공공 데이터에서 드리프트 원인을 정책/계절/이벤트/시스템 변경으로 분류한다.

KS 검정 결과를 해석하고, 단일 지표가 아닌 조합 규칙이 필요한 이유를 안다.

이 장에서 내린 네 가지 설계 결정을 민간 현장으로 옮겨, 무엇이 그대로이고 무엇이 근거만 바뀌는지 구분한다.

2.4.1 정의와 왜 위험 신호인가

데이터 드리프트는 입력 데이터의 분포가 시간에 따라 변하는 현상이다. 없으면 무엇이 실패하는가 — 모든 모델은 “학습 당시의 분포”를 전제로 동작하므로, 입력 분포가 학습 때와 달라지면 코드가 한 줄도 바뀌지 않아도 성능이 조용히 저하된다. CI는 코드 push가 있어야 실행되지만, 드리프트는 push 없이 발생한다.

그러나 드리프트 감지에는 반대 방향의 조건이 하나 붙는다 — “감지 = 항상 경보”가 아니다. 분포가 조금 변동했다고 매번 경보를 발생시키면 운영자는 알람 피로에 빠져 실제 경보까지 무시하게 된다. 그래서 드리프트는 “감지했는가”만큼이나 “이 변화가 조치가 필요한 종류인가”의 해석이 중요하다.

2.4.2 공공 데이터에서의 원인 4분류

공공 데이터에서 드리프트는 대체로 네 원인에서 발생한다.

정책 변화: 민원 유형 코드 체계가 개편되면 특정 유형의 비중이 급변한다 — 새 코드가 유입되며 분포가 계단식으로 바뀐다(2.2.4 스키마 변경과 맞물린다).

계절성: 미세먼지·한파 민원처럼 매년 반복되는 패턴이다. 갑작스러운 고장이 아니라 예측 가능한 주기이므로, 이를 “이상”으로만 해석하면 매년 같은 시기에 불필요한 재학습이 발생한다.

외부 이벤트: 대형 재난이 발생하면 접수량과 유형 분포가 일시적으로 폭증한다(버스트).

시스템 변경: 센서 교체·API 스키마 변경으로 값의 범위나 결측 패턴이 바뀐다.

민간에서도 네 갈래가 그대로 나타나되 각 항목에 해당하는 사건이 바뀐다 — 정책 변화에는 상품 카테고리 체계 개편, 계절성에는 연말 쇼핑 시즌, 외부 이벤트에는 특정 상품의 갑작스러운 수요 폭증, 시스템 변경에는 로그 수집기 버전 업그레이드가 대응한다. 다만 원인 목록 자체가 완전히 겹치지는 않는다. 정책 변화는 공공에 고유하고, 경쟁사 행동과 마케팅 캠페인은 민간에 고유하다. 그래서 드리프트 경보를 받았을 때 먼저 확인해야 할 후보 목록이 도메인마다 다르다.

2.4.3 드리프트의 유형: 급격·증분·점진·재발

변화의 시간적 양상에 따라 대응 방식도 달라진다. Lu 외(2019)의 드리프트 리뷰는 이를 네 유형으로 구분한다 — 급격(sudden: 정책 시행일에 계단식 변화)·증분(incremental: 여러 중간 상태를 거쳐 새 분포로)·점진(gradual: 옛 분포와 새 분포가 한동안 번갈아 나타나며)·재발(recurring: 계절처럼 주기적으로 돌아옴). 이 중 증분과 점진은 “서서히 변한다”는 운영 대응이 사실상 같아, 이 교재는 둘을 묶어 다룬다(11장도 같은 문헌을 같은 방식으로 소개한다). 공공에서 특히 중요한 것이 재발형이다 — 매년 여름의 변화를 매년 “이상”으로 경보하면 알람 피로만 누적된다. 재발형은 경보가 아니라 “예상된 패턴”으로 모델에 반영하는 것이 적절하다.

여기서 데이터 드리프트와 개념 드리프트(concept drift)를 구분해야 한다. 데이터 드리프트는 입력 분포가 변하는 것이고, 개념 드리프트는 입력과 정답(타겟)의 관계 자체가 변하는 것이다 — 예컨대 같은 민원 텍스트라도 정책 변화로 “긴급”의 기준이 바뀌면, 입력 분포가 그대로여도 정답이

달라진다. 이 구분은 11장에서 대응 전략과 함께 상술한다. (적용 자격: Lu 외의 분류는 드리프트 일반 이론의 배경·원칙 차용이다.)

2.4.4 감지 방법(KS 검정)과 해석

간단한 감지 방법이 **콜모고로프-스미르노프(KS) 2표본 검정**이다. 두 표본의 누적분포 사이 최대 간격(통계량 D)을 측정하고, “두 표본이 같은 분포에서 나왔다”는 귀무가설 아래 관측된 것만큼 큰 D 가 우연히 나올 확률(p -value)을 답한다. 분포 가정이 없고 연속형 값에 적합해, 두 시각의 대기질 분포 비교 같은 문제에 바로 적용할 수 있다.

해석에는 두 가지 주의가 있다. 첫째, **p -value의 의미를 정확히 파악해야 한다.** p -value는 “귀무가설이 참일 때 이만큼의 차이가 우연히 나올 확률”이지 “분포가 변했을 확률”이 아니며, 값이 크다고(비유의) 해서 “두 분포가 같다”가 증명되는 것도 아니다 — 정확히는 “다르다고 볼 충분한 증거가 없다”는 뜻이다. 게다가 p -value는 변화의 원인(계절·정책·시스템)도, 실무적으로 의미 있는 크기인지도 알려주지 않으므로, 분포 그림과 운영 맥락으로 따로 해석해야 한다. 둘째, **KS의 p -value는 표본 크기의 함수다** — 표본이 크면 사소한 차이도 유의해지고, 작으면(실습 2.2의 $n=40$) 진짜 변화도 놓칠 수 있다(검정력 부족). 그래서 현장의 드리프트 감지 도구(예: 오픈소스 Evidently)는 기준 윈도우와 비교 윈도우를 정렬해 설정하고, 데이터 특성에 맞는 검정·거리 지표를 선택하도록 설계된다. 실습 2.2처럼 변수 하나를 직접 비교하는 것은 학습용이고, 피처가 수백에서 수천 개인 현장에서는 이 비교를 자동화 도구에 맡긴다 — 민간에서는 Evidently·Whylogs 같은 오픈소스나 클라우드 사업자의 관리형 모니터링을 사용한다. 도구가 바뀌어도 판정 규칙을 정하는 일은 사람의 책임으로 남는다. (적용 자격: Evidently는 오픈소스 도구 문서 — 현장 방법 차용.)

2.4.5 단일 지표의 한계와 조합 규칙

이 두 주의를 합치면 원칙 하나가 나온다 — **단일 검정 결과만으로 판정하지 않고 조합 규칙을 사용한다.** 11장에서 이 원칙을 실제 데이터로 확인할 수 있다. 같은 서울 40개 측정소의 PM10 자료를 사용한다. 첫 번째 비교(비교 A)는 실습 2.2와 같다 — 같은 날 22:00과 23:00, 1시간 간격 비교다. 11장에서 이 비교를 다시 계산하면 KS 통계량이 0.175로 2장의 값과 동일하다. 두 번째 비교(비교 B)는 간격을 9일로 늘린다 — 2026년 6월 28일 22:00과 7월 7일 22:00을 비교한다. 이때 **PSI(Population Stability Index)는 0.3324로 경보 대역에 들어간다.** 그런데 KS는 여전히 유의하지 않다($p>0.05$). PSI(distance metric)는 경보 대역에 들어가고 KS(statistical test)는 들어가지 않는다 — 두 지표의 결론이 다르다. 이것이 지표 하나만으로 판정하면 안 되는 이유다. 11장의 운영 규칙은 두 결과를 함께 검토해 “주의” 등급으로 판정하는 것이다.

PSI에는 현장이 관행적으로 사용하는 구간이 있다 — 0.1 미만이면 안정, 0.1~0.25면 주의, 0.25 초과면 경보로 본다. 11장 실측의 0.3324가 세 번째 구간에 들어간 값이다. 여기에 지표를 하나 더 추가하면 임계 자체를 점검할 수 있다. **재학습 트리거 빈도**, 곧 드리프트 경보로 재학습이 실행된 횟수를 계수하는 것이다. 이 횟수가 지나치게 잦으면 임계가 너무 낮게 설정되었다는 뜻이므로 임계를 다시 정한다. 경보 피로를 막는 것은 해석 규칙만이 아니라 이 되먹임이다 — 공공에서는 재학습마다 검수 문서가 따라붙으므로 잦은 경보의 비용이 민간보다 오히려 크다.

민간 비즈니스 로직의 공공 적용

민간에서 두 모델의 실전 성능을 비교하는 표준은 A/B 테스트다 — 트래픽을 나눠 절반에 A 모델, 절반에 B 모델을 서비스하며 관찰한다. 그러나 공공에서 시민마다 다른 행정 서비스를 제공하는 것은 형평성·법적 문제 소지가 있으므로, 같은 목적(두 모델의 비교)을 **새도 배포(shadow deployment)**로 달성한다 — 전원에게 동일 서비스를 제공하되 후보 모델은 같은 입력을 받아 로그로만 평가한다(10장 실측). 마찬가지로 드리프트 감시 지표도 공공에서는 평균만 보지 않고 지역·집단별로 분해해 형평 측을 함께 본다 — 전체 평균이 안정적이어도 특정 지역의 분포만 변하는 것을 놓치지 않기 위해서다(16장의 그룹별 성능 분해).

공공 사례

새 민원 유형이 신설되면 특정 유형의 비중이 증가하며, 기존 모델은 “학습 당시 분포”를 전제로 동작하므로 성능이 저하될 수 있다. 또한 계절성은 갑작스러운 고장이 아니라 매년 반복되는 패턴이므로, 드리프트 감지 신호를 “이상”으로만 해석하면 불필요한 재학습이 발생할 수 있다. 따라서 드리프트는 원인 분류와 운영 의사결정(모니터링 강화/재학습/정의 변경)의 연결이 핵심이다.

역방향: 이 장의 설계를 민간 현장에 옮기면

지금까지는 민간의 방법을 공공으로 가져왔다. 반대 방향도 성립한다 — 이 장에서 정한 설계를 그대로 민간 현장에 적용하면 무엇이 유지되고 무엇이 바뀌는지 세 현장에서 확인한다.

이커머스 고객 데이터 품질 관리에서는 2.2의 다섯 위험이 이름만 바뀌 그대로 나타난다. 주문 이벤트의 중복은 이중 출고가 되고, 주소 필드의 결측은 배송 누락이 되며, 결제 모듈의 스키마 변경은 침묵 실패가 된다. 점검 방법도 그대로다 — 결측은 레코드 단위가 아니라 필드 단위로 계수해야 하고, 이름은 채워졌는데 전화번호가 빈 것과 레코드 전체가 빠진 것은 원인도 대응도 다르다(실습 2.1의 `pm10Value`와 `pm10Grade` 구분이 같은 구조다). 바뀌는 것은 임계를 정당화하는 근거다. 공공에서 결측률 임계의 근거가 “틀린 수치로 행정 결정을 내리지 않는다”였다면, 여기서는 이탈한 고객과 반품 비용이다.

실시간 광고 집계와 과금은 2.3의 시간 기준 결정이 금액으로 환산되는 현장이다. 노출·클릭 로그의 집계 기준은 이벤트 시간(클릭이 발생한 시각)이어야 하고, 처리 시간으로 집계하면 모바일 네트워크 지연 때문에 시간대별 성과가 전체적으로 어긋난다. watermark의 트레이드오프도 이 장이 설명한 그대로다 — 너무 짧으면 늦게 도착한 클릭이 폐기되어 과금이 누락되고, 너무 길면 대시보드의 지연이 커진다. 하루가 끝나면 배치로 재집계해 확정치를 산출하는 것도 같다. “빠른 근사치(스트리밍) + 정확한 확정치(배치)”라는 이중 산출 구조는 공공 보고와 민간 정산에서 동일하게 사용된다.

추천 시스템 드리프트 감시에서는 2.4의 판정 규칙이 그대로 옮겨 간다. KS와 PSI를 함께 보는 조합 규칙, 재발형을 경보가 아니라 예상된 패턴으로 다루는 원칙 모두 유효하다. 특히 후자는 민간에서 더 중요하다 — 해마다 같은 쇼핑 시즌에 같은 경보가 발생하면 운영팀이 경보 자체를 무시하기 시작하기 때문이다.

공통 원칙. 이 장이 내린 네 가지 결정 — 데이터를 유입 패턴으로 분류할 것, 위험마다 운영 신호와 최소 대응을 짚지을 것, 집계 기준을 이벤트 시간으로 설정할 것, 드리프트를 단일 지표가 아니라 조합 규칙으로 판정할 것 — 은 어느 도메인에서도 똑같이 내려야 한다. **근거만 도메인에 따라 달라진다.** 공공에서 그 근거가 감사·형평성·법적 의무라면, 민간에서는 매출·비용·SLA다. 한 가지는 방향이 비대칭이다. 민간 방법을 공공에 도입하면 증거 산출 의무가 한 겹 추가되지만, 공공 설계를 민간에 도입하면 그 의무를 제외해도 손실이 없다 — 필드별 결측 집계는 품질 리포트가 되고, 조합 규칙은 불필요한 재학습 비용을 줄인다.

핵심 정리

드리프트는 감지(통계/지표)와 해석(원인/맥락)이 함께 있어야 의미가 있으며, “감지 = 항상 경보”가 아니다.

공공 데이터에서는 반복 패턴(계절성)과 구조 변화(정책/시스템)를 구분해야 한다.

단일 지표에 의존하지 않고 조합 규칙을 사용한다 — KS와 PSI의 판정이 서로 다른 실측(11장)이 그 근거다.

유형 분류·위험별 지표·이벤트 시간 기준·조합 판정이라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 2.1 공개 API 응답(JSON) 파싱과 기본 EDA(★★)

실습 목표

에어코리아 응답의 측정소 레코드를 DataFrame으로 변환한다.

결측 표기(-)를 NaN으로 바꾸고 결측률·분포·등급을 **필드별로** 요약한다.

실행

```
cd practice/chapter2
pip install -r code/requirements.txt
python3 code/2-1-parse-api.py # 저장된 22:00 스냅샷 EDA
# 실시간으로 보려면: python3 code/2-1-parse-api.py --live --sido 서울
```

핵심 코드

```
df = pd.DataFrame(items) # response.body.items
df[col] = pd.to_numeric(df[col].replace({"-": np.nan}), errors="coerce")
summary = run_eda(df) # 결측·분포·등급
```

전체 코드는 `practice/chapter2/code/2-1-parse-api.py` 참고

실행 결과(실제)

서울 22:00 스냅샷(측정소 40개)에 대한 실제 EDA 결과다(source: "file:airquality_seoul_2200.json").

출력 파일: practice/chapter2/data/output/ch2_eda_summary.json

측정소 수: 40개

PM10(미세먼지): 최소 14, 최대 27, 평균 20.0, 중앙값 20.0 ($\mu\text{g}/\text{m}^3$)

PM2.5: 최소 6, 최대 17, 평균 12.7, 중앙값 12.5 ($\mu\text{g}/\text{m}^3$)

PM10(미세먼지) 측정값 결측: 0건. 그러나 등급(pm10Grade , 미세먼지 등급)은 1건 결측 — 값은 있으나 파생 등급이 비었다.

PM10 등급 분포: 1등급(좋음) 28개, 2등급(보통) 11개, 결측 1개

관찰

측정값은 모두 채워졌지만 등급이 1건 비었다. “값 결측”과 “파생 필드 결측”은 별개이며 (2.2.3), 결측 점검은 필드별로 따로 해야 한다. 만약 결측을 데이터 전체 단위로만 계수했다면 “결측 0건”이라는 잘못된 판단에 이르렀을 것이다.

이 결측 레코드(강남구, 22:00)는 pm10Flag 도 비어 있다 — 2.2.3에서 소개한 “플래그로 결측 사유를 구분한다”는 방법이 이 사례에는 적용되지 않는다. 플래그가 모든 결측의 원인을 제시하지는 않으며, 원인 불명 결측이 남는 것도 실무에서 발생하는 현실이다.

이 스냅샷은 한 시각(22:00)의 공간 분포다. 시간에 따른 변화(드리프트)는 실습 2.2에서 두 시각을 비교한다.

검수 포인트(발주 요구사항으로 전환)

이 관찰을 1장 1.4.4의 원칙대로 “검수 가능한 요구사항 문장”으로 바꾼다.

“결측 점검은 데이터 전체가 아니라 필드별로 수행하고, 값 필드와 파생 필드(등급 등)의 결측을 분리 집계해 지표로 노출할 것.”

“결측 표기(- / null)를 0으로 대체하지 않고 NaN으로 처리하며, 그 규칙을 문서화할 것.”

“산출물에 측정 시각(dateTime)·출처(source)를 기록해 재현 가능성을 확보할 것.”

실습 2.2 단일 변수 분포 변화 시각화(★★)

실습 목표

같은 측정 변수(PM10)의 두 시각 분포(기준 vs 현재)를 비교한다.

KS 2표본 검정과 분위수·히스토그램으로 “값은 변해도 분포 변화가 통계적으로 유의하지 않을 수 있다”는 결과와 그 해석을 익힌다.

실행

두 스냅샷은 같은 API를 정시마다 호출해 수집한 것이다(기준 22:00, 현재 다음 정시).

```
cd practice/chapter2
python3 code/2-2-drift-visualize.py \
  --baseline data/input/airquality_seoul_2200.json \
  --current data/input/airquality_seoul_current.json \
  --col pm10Value
```

핵심 코드

```
base, base_time = load_series(Path(baseline), col) # 기준 스냅샷
curr, curr_time = load_series(Path(current), col)  # 현재 스냅샷
stat, p = ks_2samp_with_fallback(base, curr)      # 분포 동일성 검정
```

전체 코드는 `practice/chapter2/code/2-2-drift-visualize.py` 참고

실행 결과(실제)

같은 서울 측정소의 PM10을 **22:00(기준)**과 **23:00(현재)** 두 실측 스냅샷으로 비교한 결과다. 출력은 `ch2_drift_result.json` (검정)과 `ch2_drift_visual.txt` (분위수·히스토그램)에 저장된다.

기준 22:00: 40개 측정소, 평균 $20.0\mu\text{g}/\text{m}^3$

현재 23:00: 40개 측정소, 평균 $20.8\mu\text{g}/\text{m}^3$

KS 통계량 0.175, p-value 0.579 ($\alpha=0.05 \rightarrow$ **drift_detected = False**)

분위수 이동: 25%는 $17 \rightarrow 19$, 75%는 $22 \rightarrow 23$ 으로 소폭 상승(50% 중앙값은 $20 \rightarrow 20$ 으로 동일)

표 2.4 PM10 분위수 비교(기준 22:00 vs 현재 23:00)

분위수	기준(22:00)	현재(23:00)
0%	14.0	14.0
25%	17.0	19.0
50%	20.0	20.0
75%	22.0	23.0
100%	27.0	27.0

해석: '값은 변하고 구조는 같다' — 유의한 분포 변화는 검출되지 않았다는 뜻

한 시간 사이 평균이 20.0에서 20.8로 상승했고 분포도 소폭 위로 이동했지만, KS 검정의 p-value(0.579)는 0.05보다 훨씬 커서 "두 분포가 다르다"고 말할 통계적 근거가 부족하다. 여기서 표현에 주의할 점이 있다(2.4.4) — 이것은 "두 분포가 같음"을 증명한 것이 아니라 "다르다고 볼 증거가 부족하다"는 뜻이고, $n=40$ 의 소표본이라 작은 변화는 애초에 검출하기 어렵다는 검정력의 한계도 함께 작용한다. 절의 제목 "값은 변하고 구조는 같다"는 이 실습 수준에서 관찰을 요약한 교육적 표현이지 통계적 동일성 증명이 아니다. 그럼에도 실무적 교훈은 분명하다 — 짧은 간격의 정상적 변동을 매년 경보로 처리하지 않는다는 것("드리프트 감지=항상 경보"가 아님, 2.4.1). 만약 며칠~몇 주 간격으로 계절이나 정책이 바뀌면 PSI 같은 거리 지표나 구간별 이동이 먼저 커지고 (11장 비교 B의 PSI 0.3324처럼 — 이때도 KS는 여전히 비유의일 수 있다), 그런 신호들을 조합해 판단할 때 비로소 재학습·정의 변경 같은 운영 의사결정으로 이어진다.

해석 포인트

시간 간격이 짧으면 분포 변화가 작아 드리프트가 '없음'으로 나올 수 있다. 이것이 정상이며, 이 짧은 간격 비교(비교 A)는 11장에서 그대로 재현되어(KS 0.175 일치) 감시 코드 자체의 적합성 검증에 사용된다.

KS p-value는 귀무가설 아래 관측된 차이가 얼마나 드문지를 나타낼 뿐, "두 분포가 다를 확률"도 변화의 원인(계절·정책·시스템)도 제시하지 않는다(2.4.4). 원인 해석은 분포 그림과 운영 맥락으로 보완한다.

$n=40$ 의 소표본이라 검정력이 낮다는 점도 기억해야 한다 — 이 실습의 관찰 대상은 대기질 분석이 아니라 분포 비교·판정 절차다. 같은 데이터에 거리 지표(PSI)를 더해 조합 규칙으로 판정하는 운영형 확장이 11장이다.

검수 포인트(발주 요구사항으로 전환)

드리프트 감시를 "실시간 처리를 지원할 것" 같은 검수 불가능한 문장이 아니라, 검수 가능한 요구사항으로 바꾼다(1장 1.4.4).

"드리프트 판정은 단일 지표가 아니라 통계 검정(KS)과 거리 지표(PSI)의 조합 규칙으로 하고, 두 신호의 판정이 다를 때 적용할 등급 판정 규칙을 문서화할 것."

"기준 윈도우와 비교 윈도우를 같은 조건(같은 시각·요일·집계 단위)으로 정렬해 비교하고, 정렬 규칙을 산출물에 기록할 것."

"표본 크기(n)와 검정력의 한계를 판정 리포트에 명시하고, 소표본에서의 비검출을 '변화 없음'으로 단정하지 않을 것."

실습 확장

--col 을 pm25Value 로 바꿔 같은 두 시각을 비교하고, PM10과 결과가 어떻게 다른지 관찰한다.

두 스냅샷의 시간 간격을 벌려(예: 하루·1주) 다시 비교하면, 짧은 간격에서 '없음'이던 판정이 언제 전환되는지 확인한다 — 이 확장을 실제 9일 간격으로 수행한 것이 11장의 비교 B다.

핵심 용어

용어	뜻	이 책에서 실측으로 다루는 곳
유입 패턴	데이터가 들어오는 형태(지속·버스트·고빈도·주기 갱신)	2장(유형 분류), 6장(버스트·지연)
멱등성(Idempotency)	같은 작업을 여러 번 수행해도 결과가 한 번과 같은 성질	5장(UPSERT), 7장(재실행 해시 동일)
이벤트 시간과 처리 시간	사건 발생 시간과 시스템 수신·처리 시각의 구분	2장·6장(watermark 수용·폐기 실측)
침묵 실패(Silent failure)	중단되지 않은 채 틀리거나 빈 결과를 계속 산출하는 실패	4·6·7·8장(유실·폐기·drop·빈 적재)
데이터 계약(Data contract)	생산자-소비자 간 스키마·의미·품질·변경 절차의 합의	13장(카탈로그·소비자·변경관리)
데이터 드리프트	입력 데이터 분포의 시간적 변화	2장(KS 0.175 비검출), 11장(PSI 0.3324 경보·조합 규칙)
개념 드리프트	입력과 타겟 관계 자체의 변화	11장(개념 구분)
데이터 관측성	적시성·양·스키마·분포·계보의 지속 감시	2장(개념), 11장(모니터링 실측)

다음 장 예고

다음 장에서는 공공 영역에서 DE-MLOps를 실제로 결합하기 위한 아키텍처 설계 관점을 다룬다. 특히 이 장에서 분류한 품질 위험과 계약이 데이터 계약·메타데이터·관측성이라는 “구성 요소”로 어떻게 연결되는지 확장한다.

참고문헌

Akidau, T., Bradshaw, R., Chambers, C., et al. (2015). *The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing*. VLDB. <https://www.vldb.org/pvldb/vol8/p1792-Akidau.pdf>
Breck, E., Zinkevich, M., Polyzotis, N., Whang, S., & Roy, S. (2019). *Data Validation for Machine Learning*. MLSys. <https://mlsys.org/Conferences/2019/doc/2019/167.pdf>
Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363. <https://doi.org/10.1109/TKDE.2018.2876857> (프린트 arXiv:2004.05785 —

<https://arxiv.org/abs/2004.05785>)

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.

<https://dataintensive.net/>

Evidently AI. (n.d.). *What is data drift in ML, and how to detect and handle it*.

<https://www.evidentlyai.com/ml-in-production/data-drift>

Monte Carlo. (n.d.). *What is Data Observability? The Five Pillars*.

<https://www.montecarlodata.com/blog-what-is-data-observability/>

ThoughtWorks. (n.d.). *Data Contracts*. <https://www.thoughtworks.com/insights/blog/data-strategy/data-contracts-a-bridge-connecting-decentralized-teams>

Data Contract Specification. (n.d.). <https://datacontract.com/>

Confluent. (n.d.). *Schema Evolution and Compatibility for Schema Registry*.

<https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>

개인정보보호법. 국가법령정보센터. <https://www.law.go.kr/> (제18조 목적 외 이용·제공 제한, 제23조 민감정보 처리 제한)

개인정보보호위원회. <https://www.pipc.go.kr/>

한국환경공단 에어코리아. *대기오염정보*. <https://www.airkorea.or.kr/>

Public Data Portal (Korea). <https://www.data.go.kr/>

KOSIS (Korean Statistical Information Service). <https://kosis.kr/>

Apache Kafka. *Documentation*. <https://kafka.apache.org/documentation/>

SciPy. *scipy.stats.ks_2samp*.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

pandas. *Documentation*. <https://pandas.pydata.org/docs/>

Matplotlib. <https://matplotlib.org/stable/>